

Documentação do Desenvolvimento

1. Identificação do projeto

Projeto: Cidadania na Escola: informação de qualidade também é conquista

Grupo: Código Cidadão

Tecnologias: Angular, TypeScript, Django, Django REST Framework e PostgreSQL.

O projeto foi desenvolvido como parte de uma atividade de extensão, com o objetivo de utilizar a tecnologia como ferramenta de apoio à educação, cidadania e disseminação de informação de qualidade.

A proposta envolve temas como democracia, cidadania, desinformação, verificação de informações e direitos trabalhistas, buscando apresentar os conteúdos de maneira simples e acessível, principalmente para estudantes da Educação de Jovens e Adultos (EJA).

2. Organização inicial do projeto

O projeto foi organizado utilizando uma arquitetura separando o **frontend**, responsável pela interface visual, do **backend**, responsável pelas regras de negócio e disponibilização dos dados.

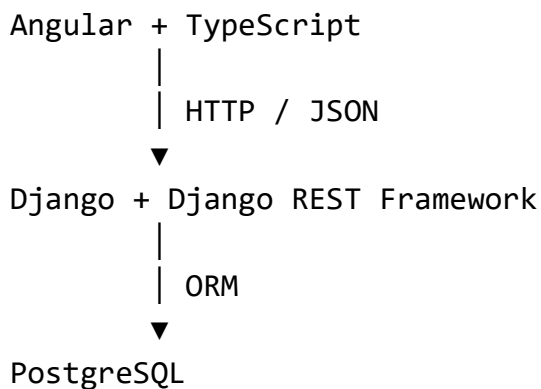
A estrutura planejada é:

```
Cidadania-na-Escola/
|
|— backend/
|   |— api/
|   |— config/
|   |— manage.py
|   |— venv/
|
|— frontend/
|   |— src/
|   |— angular.json
|   |— package.json
```

```
|  └─ ...
|
├─ .gitignore
└─ README.md
```

Essa separação facilita a manutenção e permite que cada parte do sistema seja desenvolvida de forma independente.

A comunicação entre as partes será realizada através de uma API utilizando HTTP e JSON:



3. Criação do repositório no GitHub

Foi criado um repositório no GitHub para armazenar o código-fonte e acompanhar a evolução do projeto.

O repositório utilizado é:

[Cidadania-na-Escola no GitHub](#)

O projeto foi clonado localmente utilizando:

```
git clone https://github.com/marcoslima2207/Cidadania-na-Escola.git
```

Após o clone, o projeto passou a ser desenvolvido localmente dentro de:

C:\Users\Marcos Eduardo\Cidadania-na-Escola

4. Configuração do ambiente Python

Para o desenvolvimento do backend foi utilizado Python com um ambiente virtual.

Primeiro, foi acessada a pasta backend:

```
cd "C:\Users\Marcos Eduardo\Cidadania-na-Escola\backend"
```

Depois foi criado o ambiente virtual:

```
python -m venv venv
```

O ambiente foi ativado com:

```
.\venv\Scripts\Activate.ps1
```

O uso do ambiente virtual permite manter as dependências do projeto isoladas das demais instalações de Python existentes no computador.

5. Instalação das dependências do Django

Depois da criação do ambiente virtual, foram instaladas as principais dependências utilizadas pelo backend:

```
pip install django djangorestframework django-cors-headers psycopg
```

As principais bibliotecas possuem as seguintes funções:

Tecnologia	Função
Django	Framework utilizado para desenvolvimento do backend
Django REST Framework	Criação da API REST
django-cors-headers	Permite a comunicação entre frontend e backend em diferentes portas
psycopg	Driver utilizado para comunicação entre Django e PostgreSQL
Python	Linguagem utilizada no backend

Durante a configuração houve inicialmente um problema relacionado ao driver do PostgreSQL. O problema foi solucionado com a instalação/configuração do `psycopg`, permitindo que o Django se comunicasse corretamente com o banco.

A versão instalada do `psycopg` verificada foi:

3.3.5

6. Criação do projeto Django

Com o ambiente preparado, foi criado o projeto Django utilizando:

```
django-admin startproject config .
```

O projeto passou a possuir uma estrutura semelhante a:

```
backend/
|
|— config/
|   |— __init__.py
|   |— settings.py
|   |— urls.py
|   |— asgi.py
|   |— wsgi.py
|
|— manage.py
|— venv/
```

O diretório `config` concentra as configurações principais do projeto Django.

7. Criação da aplicação API

Em seguida foi criada a aplicação responsável pela API:

```
python manage.py startapp api
```

A aplicação `api` ficou responsável por concentrar os modelos, serializers, views e URLs relacionadas aos dados do projeto.

Sua estrutura passou a incluir:

```
api/  
├── migrations/  
├── __init__.py  
├── admin.py  
├── apps.py  
├── models.py  
├── tests.py  
└── views.py
```

Posteriormente também foi criado o arquivo:

```
serializers.py
```

8. Configuração do Django

O arquivo:

```
backend/config/settings.py
```

foi configurado para utilizar as aplicações necessárias.

Entre elas:

```
'rest_framework',  
'corsheaders',  
'api',
```

Também foi configurado o middleware do CORS:

```
'corsheaders.middleware.CorsMiddleware',
```

Isso será necessário porque o Angular será executado normalmente em:

<http://localhost:4200>

enquanto o Django será executado em:

<http://127.0.0.1:8000>

Foi configurado:

```
CORS_ALLOWED_ORIGINS = [  
    'http://localhost:4200',  
]
```

Assim, o frontend poderá realizar requisições para a API durante o desenvolvimento.

9. Configuração do PostgreSQL

O banco de dados escolhido para o projeto foi o **PostgreSQL**.

A versão instalada no computador é:

PostgreSQL 18.6

O executável `psql` está localizado em:

`C:\Program Files\PostgreSQL\18\bin\psql.exe`

Como o PostgreSQL não estava configurado no PATH do Windows, foi utilizado o caminho completo para acessar o terminal do banco:

```
& "C:\Program Files\PostgreSQL\18\bin\psql.exe" -U postgres
```

Foi verificada a existência do banco:

```
cidadania_escola
```

A conexão com o banco foi realizada utilizando:

```
\c cidadania_escola
```

Depois, o terminal do PostgreSQL foi encerrado com:

```
\q
```

10. Configuração da conexão Django + PostgreSQL

O Django foi configurado para utilizar o PostgreSQL em vez do banco SQLite padrão.

A configuração utiliza variáveis de ambiente:

```
DB_NAME
DB_USER
DB_PASSWORD
DB_HOST
DB_PORT
```

Por exemplo:

```
$env:DB_NAME="cidadania_escola"
$env:DB_USER="postgres"
$env:DB_PASSWORD="SUA_SENHA"
$env:DB_HOST="localhost"
$env:DB_PORT="5432"
```

A senha real não foi colocada diretamente no código nem compartilhada.

Essa abordagem também facilita a utilização de diferentes configurações de banco em ambientes de desenvolvimento, testes e produção.

11. Teste da configuração do Django

Após a configuração do banco e das dependências, foi executado:

```
python manage.py check
```

O resultado foi:

```
System check identified no issues (0 silenced).
```

Isso confirmou que, naquele momento, não havia problemas detectados na configuração do projeto Django.

Também foi verificada a versão do driver PostgreSQL:

```
python -c "import psycopg; print(psycopg.__version__)"
```

Resultado:

```
3.3.5
```

12. Criação do modelo de conteúdo

Foi criado o primeiro modelo do sistema no arquivo:

backend/api/models.py

O modelo representa os conteúdos educativos disponibilizados pela plataforma.

```
class Conteudo(models.Model):
    CATEGORIAS = [
        ('democracia', 'Democracia e Cidadania'),
        ('desinformacao', 'Desinformação'),
        ('checagem', 'Checagem de Informação'),
        ('trabalhista', 'Direitos Trabalhistas'),
    ]

    titulo = models.CharField(max_length=200)
    descricao = models.TextField()
    categoria = models.CharField(max_length=30, choices=CATEGORIAS)
    conteudo = models.TextField()
    data_publicacao = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.titulo
```

O modelo possui:

- título;
- descrição;
- categoria;
- conteúdo;
- data de publicação.

As categorias definidas são:

- Democracia e Cidadania;
- Desinformação;
- Checagem de Informação;
- Direitos Trabalhistas.

13. Migrations

Depois da criação do modelo, foi criada a migration:

```
python manage.py makemigrations
```

O Django identificou a criação do modelo:

```
Migrations for 'api':  
  api\migrations\0001_initial.py  
  + Create model Conteudo
```

Depois a migration foi aplicada ao PostgreSQL:

```
python manage.py migrate
```

O resultado confirmou:

```
Applying api.0001_initial... OK
```

As migrations permitem que a estrutura dos modelos Django seja transformada em estruturas correspondentes no banco de dados.

14. Criação do Serializer

Para transformar os objetos Django em dados JSON, foi criado:

```
backend/api/serializers.py
```

Com:

```
from rest_framework import serializers
```

```
from .models import Conteudo
```

```
class ConteudoSerializer(serializers.ModelSerializer):  
    class Meta:  
        model = Conteudo  
        fields = [  
            'id',
```

```
        'titulo',
        'descricao',
        'categoria',
        'conteudo',
        'data_publicacao',
    ]
```

O serializer faz a conversão entre os objetos do Django e o formato JSON utilizado pela API.

15. Criação da ViewSet

Foi criada a ViewSet responsável pelos conteúdos:

backend/api/views.py

Código:

```
from rest_framework import viewsets

from .models import Conteudo
from .serializers import ConteudoSerializer


class ConteudoViewSet(viewsets.ModelViewSet):
    queryset = Conteudo.objects.all().order_by('-data_publicacao')
    serializer_class = ConteudoSerializer
```

O uso do ModelViewSet permite disponibilizar operações da API para o modelo Conteudo, como:

- listar;
- consultar;
- criar;
- atualizar;
- excluir.

16. Configuração das URLs da API

Foi criado:

`backend/api/urls.py`

Utilizando o `DefaultRouter` do Django REST Framework:

```
from django.urls import include, path

from rest_framework.routers import DefaultRouter

from .views import ConteudoViewSet

router = DefaultRouter()

router.register(r'conteudos', ConteudoViewSet, basename='conteudo')

urlpatterns = [
    path('', include(router.urls)),
]
```

Também foi configurado o arquivo:

`backend/config/urls.py`

para disponibilizar a API:

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('api.urls')),
]
```

Com isso, o endpoint principal ficou definido como:

`/api/conteudos/`

Durante o desenvolvimento, a URL esperada para acesso é:

<http://127.0.0.1:8000/api/conteudos/>

17. Criação do frontend Angular

Depois da preparação do backend, foi criado o frontend com Angular.

A criação foi realizada a partir da raiz do projeto:

```
ng new frontend
```

Durante a criação foram selecionadas as seguintes opções:

- Angular;
- TypeScript;
- CSS;
- sem SSR/SSG;
- sem ferramenta adicional de IA.

O frontend foi criado em:

C:\Users\Marcos Eduardo\Cidadania-na-Escola\frontend

18. Execução do Angular

O frontend foi executado utilizando:

```
ng serve
```

O Angular iniciou corretamente e apresentou:

```
Application bundle generation complete.
```

```
Watch mode enabled.
```

```
Watching for file changes...
```

```
→ Local: http://localhost:4200/
```

Isso confirmou que a aplicação Angular estava funcionando localmente.

19. Desenvolvimento da primeira interface

A página inicial padrão do Angular foi substituída por uma interface personalizada para o projeto.

A página contém elementos como:

Cabeçalho

Com o nome:

Cidadania na Escola

E opções de navegação:

- Início
- Democracia
- Desinformação
- Direitos Trabalhistas
- Quiz
- Sobre

Área principal

Foi criada uma apresentação do projeto:

CÓDIGO CIDADÃO

Com a mensagem:

Informação de qualidade. Cidadania mais forte.

Também foi adicionado o botão:

Começar agora

Cards de conteúdo

Foram criados cards para apresentar os principais temas:

- Democracia e Cidadania;

- Desinformação;
- Direitos Trabalhistas.

Também foi adicionado um rodapé.

20. Identidade visual

A interface foi desenvolvida seguindo uma identidade visual baseada principalmente em tons de azul e verde.

As principais cores utilizadas foram:

#153E79
#23A345
#E2F6F1
#244467
#7190A0

A escolha busca transmitir uma aparência relacionada a:

- educação;
- confiança;
- cidadania;
- tecnologia;
- informação.

21. Responsividade

A interface inicial também recebeu regras CSS para adaptação a diferentes tamanhos de tela.

Foi utilizada uma estrutura baseada em cards e grid, permitindo que os conteúdos sejam reorganizados em telas menores.

Isso é importante porque a plataforma poderá ser acessada tanto por computadores quanto por dispositivos móveis.

22. Configuração do .gitignore

Como o projeto possui duas tecnologias diferentes, o .gitignore foi adaptado para proteger arquivos que não devem ser enviados para o GitHub.

Foram mantidas as regras do Angular, como:

```
/node_modules
/frontend/dist/
/frontend/.angular/
```

E adicionadas regras para Python/Django:

```
__pycache__/
*.py[cod]
venv/
.venv/
db.sqlite3
*.log
.env
.env.*
*.sql
```

Também foram mantidas regras para arquivos do VS Code e do sistema operacional.

Essa configuração evita enviar para o repositório arquivos desnecessários, ambientes virtuais, dependências instaladas e informações que podem ser sensíveis.

23. Controle de versão com Git

Depois da configuração do projeto, foi verificado o estado do repositório:

```
git status
```

Inicialmente o Git informou:

```
On branch main
Your branch is up to date with 'origin/main'.
```

Untracked files:

```
backend/  
frontend/
```

Isso significa que o Git reconhecia o repositório, mas ainda não estava acompanhando os arquivos dos projetos backend e frontend.

Foi utilizado:

```
git add .
```

para adicionar os arquivos ao próximo commit.

Depois foi realizada a verificação:

```
git status
```

E, posteriormente, foi preparado o commit:

```
git commit -m "feat: estrutura inicial angular e django"
```

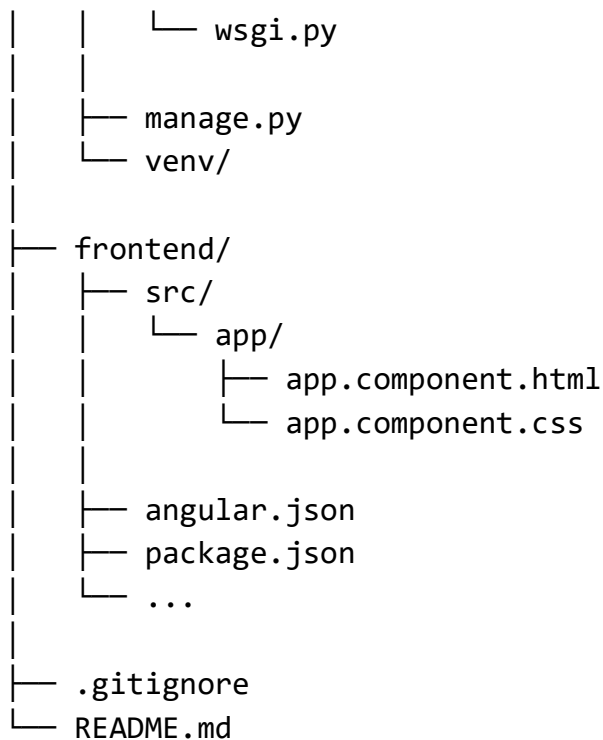
O envio para o GitHub foi realizado com:

```
git push origin main
```

24. Estado atual do projeto

Até este ponto, o projeto possui a seguinte base:

```
Cidadania-na-Escola/  
|  
├── backend/  
│   ├── api/  
│   │   ├── migrations/  
│   │   ├── models.py  
│   │   ├── serializers.py  
│   │   ├── views.py  
│   │   └── urls.py  
│   ├── config/  
│   │   ├── settings.py  
│   │   ├── urls.py  
│   │   └── asgi.py
```



25. O que já está funcionando

Backend

- ✓ Python configurado
- ✓ Ambiente virtual criado
- ✓ Django instalado
- ✓ Django REST Framework configurado
- ✓ CORS configurado
- ✓ PostgreSQL instalado
- ✓ Banco `cidadania_escola` criado
- ✓ Django conectado ao PostgreSQL
- ✓ Modelo Conteudo criado
- ✓ Migration criada
- ✓ Migration aplicada
- ✓ Serializer criado
- ✓ ViewSet criada
- ✓ Rotas da API configuradas

Frontend

- ✓ Angular instalado/configurado
- ✓ Projeto criado
- ✓ TypeScript configurado pelo Angular
- ✓ CSS configurado
- ✓ Página inicial personalizada
- ✓ Identidade visual criada
- ✓ `ng serve` funcionando
- ✓ Aplicação disponível em `localhost:4200`

Git/GitHub

- ✓ Repositório criado
- ✓ Projeto clonado
- ✓ `.gitignore` configurado
- ✓ Backend preparado para versionamento
- ✓ Frontend preparado para versionamento
- ✓ Estrutura preparada para commit e push

26. Próximas etapas

O que temos agora é a **estrutura inicial funcional**. As próximas etapas podem ser desenvolvidas progressivamente:

FASE 1 – Estrutura



Angular + Django + PostgreSQL



FASE 2 – API



Mais modelos e endpoints



FASE 3 – Frontend



Integração Angular ↔ Django



FASE 4 – Conteúdo



Democracia / Desinformação /
Checagem / Direitos Trabalhistas



FASE 5 – Quiz



Perguntas + respostas + pontuação



FASE 6 – Testes



Testes do backend e frontend



FASE 7 – Segurança



Validações + permissões + proteção de dados



FASE 8 – Apresentação



Intervenção na escola EJA

Resumo

Durante a primeira etapa do desenvolvimento, foi criada a estrutura inicial do projeto Cidadania na Escola utilizando Angular no frontend, Django e Django REST Framework no backend e PostgreSQL como banco de dados. O repositório foi configurado no GitHub para controle de versão e organização do código.

No backend, foi criado um ambiente virtual Python, instaladas as dependências necessárias e configurado o projeto Django. Também foi criada a aplicação `api`, configurada a comunicação com o PostgreSQL e desenvolvido o primeiro modelo de conteúdo da plataforma. Através do Django REST Framework, foram implementados serializadores, ViewSets e rotas para disponibilizar os conteúdos por meio de uma API REST.

No frontend, foi criado um projeto Angular utilizando TypeScript e CSS. A página inicial padrão foi personalizada de acordo com a proposta do projeto, apresentando os temas de democracia, desinformação e direitos trabalhistas, além da identidade visual do grupo Código Cidadão.

Por fim, o arquivo `.gitignore` foi configurado para evitar o versionamento de arquivos desnecessários, como ambientes virtuais Python, `node_modules`, arquivos

temporários e variáveis de ambiente. Dessa forma, foi estabelecida a base tecnológica necessária para as próximas etapas de desenvolvimento e integração da plataforma.

ETAPAS QUE FALTAM CONCLUIR

1. Finalizar o Backend/API

Já temos Conteúdo, mas ainda precisamos criar os outros recursos planejados:

- Usuario
- Pergunta
- Resultado
- serializers
- views/viewsets
- URLs/endpoints
- validações
- testar os endpoints

A API ficará aproximadamente assim:

```
/api/conteudos/  
/api/perguntas/  
/api/resultados/  
/api/usuarios/
```

2. Criar o banco completo

Hoje temos a estrutura inicial de Conteúdo.

Precisamos chegar a algo próximo de:

```
PostgreSQL  
|  
├── usuarios  
├── conteudos  
├── perguntas  
└── resultados
```

Depois:

```
python manage.py makemigrations  
python manage.py migrate
```

E cadastrar alguns conteúdos reais para testar a aplicação.

3. Fazer Angular conversar com Django

Essa é uma das partes **mais importantes que ainda faltam**.

Hoje o Angular funciona visualmente e o Django possui a API, mas precisamos conectar os dois:

```
Angular  
|  
| GET / POST / PUT / DELETE  
▼  
Django REST API  
|
```

▼
PostgreSQL

No Angular teremos, por exemplo:

```
services/  
└─ conteudo.service.ts
```

Esse serviço buscará os conteúdos da API.

Assim, em vez de deixar os textos diretamente no HTML, o Angular poderá receber:

```
{  
  "id": 1,  
  "titulo": "O que é democracia?",  
  "descricao": "Entenda o conceito...",  
  "categoria": "democracia"  
}
```

4. Criar as páginas do sistema

A página inicial já começou a ser feita. Ainda precisamos criar as páginas/seções:

```
Início  
├─ Democracia  
├─ Desinformação  
├─ Checagem  
├─ Direitos Trabalhistas  
├─ Quiz  
└─ Sobre o Projeto
```

E configurar as rotas do Angular.

Por exemplo:

```
/  
/democracia  
/desinformacao  
/checagem  
/direitos-trabalhistas  
/quiz  
/sobre
```

5. Implementar o Quiz

Essa será uma das funcionalidades principais.

Precisaremos:

Perguntas



Alternativas



Usuário responde



Sistema verifica



Calcula pontuação



Mostra resultado

Exemplo:

Pergunta 1

O que caracteriza uma informação confiável?

- ☐ A) ...
- ☐ B) ...
- ☐ C) ...
- ☐ D) ...

[Responder]

Ao final:

Quiz concluído!

Sua pontuação: 8/10

O resultado poderá ser enviado para:

POST /api/resultados/

6. Implementar a parte de desinformação e checagem

Precisamos criar conteúdos educativos sobre:

- o que é desinformação;
- como identificar informações suspeitas;
- importância de verificar fontes;
- diferença entre opinião e informação;
- cuidados antes de compartilhar uma notícia;
- exemplos educativos de situações de desinformação.

A área de **Checagem** pode ensinar um passo a passo:

1. Verifique a fonte
↓
2. Confira a data
↓
3. Procure a informação em outras fontes
↓
4. Analise o contexto
↓
5. Só então compartilhe

7. Implementar Direitos Trabalhistas

Essa parte precisa ser feita com bastante cuidado.

Podemos criar uma área educativa com conteúdos como:

Direitos Trabalhistas

- |
- |— Contrato de trabalho
- |— Jornada de trabalho
- |— Férias
- |— 13º salário
- |— FGTS
- |— Outros conteúdos definidos pelo projeto

O conteúdo deve ser apresentado como **material educativo**, sem substituir orientação jurídica profissional.

8. Testar e preparar a entrega

Depois de implementar tudo, precisamos fazer uma etapa de testes.

Backend

Testar:

GET
POST
PUT
DELETE

dos endpoints.

Frontend

Testar:

- navegação;
- formulários;
- quiz;
- carregamento dos conteúdos;
- responsividade;
- mensagens de erro.

Integração

Testar:

Angular → Django → PostgreSQL

e:

PostgreSQL → Django → Angular

O que falta no projeto inteiro

Se eu colocasse nosso progresso em um checklist:

Parte	Situação
-------	----------

Planejamento	☒
Definição da tecnologia	☒
GitHub	☒
.gitignore	☒
Django	☒
Django REST Framework	☒
PostgreSQL	☒
Conexão Django + PostgreSQL	☒
Modelo Conteúdo	☒
API de conteúdos	☒
Angular	☒
Página inicial	☒
Identidade visual	☒
API → Angular	☐
Usuários	☐
Perguntas	☐
Resultados	☐
Quiz	☐
Página Democracia	☐
Página Desinformação	☐
Página Checagem	☐
Direitos Trabalhistas	☐
Página Sobre	☐
Testes	☐
Revisão de segurança	☐
Documentação final	☐
Preparação para apresentação	☐

Ordem que eu recomendo seguirmos

Para não virar uma bagunça, eu faria exatamente nesta sequência:

- 1** Completar os modelos do Django

↓
- 2** Completar a API REST

↓
- 3** Testar a API

